Control 1 CC4303
Parte 2 — DNS Resolver

Integrante: Julián Proboste

Introducción

Un DNS resolver recibe consultas DNS de clientes y las resuelve siguiendo la jerarquía del sistema de nombres de dominio. Cuando un cliente pregunta por un dominio, el resolver comienza consultando a los root servers, los cuales no conocen la IP final pero saben quién administra cada zona. Desde ahí el resolver sigue las delegaciones hasta llegar al servidor de nombre autoritativo que posee la IP del dominio consultado. El resolver implementado en esta actividad utiliza el algoritmo iterativo, donde él se encarga de realizar cada consulta sucesiva a los distintos Name Servers hasta obtener la respuesta.

Para construir el resolver se utilizaron las librerías `dnslib` y `socket` de Python. La primera se encarga de crear y parsear mensajes DNS, ocultando el formato binario del protocolo. La segunda provee los sockets necesarios para la comunicación con los servidores DNS.

El resolver corre en una máquina virtual, anclado a su IP y escuchando en el puerto 8000. Desde la máquina local se accede a él con `dig -p8000 @IP_VM`. A lo largo de este informe se documenta cada paso de la implementación, las pruebas de funcionalidad y los experimentos solicitados.

Paso 1. Socket receptor

El primer paso es crear un socket que reciba mensajes DNS. Se utiliza un socket `SOCK_DGRAM` porque el protocolo DNS opera sobre UDP en el puerto 53. El socket se asocia a la dirección `(IP_VM, 8000)` usando el puerto 8000 en vez de 53, ya que el puerto 53 es un puerto reservado por el sistema operativo para el servicio DNS del propio sistema.

El socket queda en un loop infinito recibiendo mensajes con `recvfrom(4096)`. El buffer de 4096 bytes es suficiente para la mayoría de las consultas DNS y no es necesario preocuparse por mensajes más grandes que el buffer en esta actividad. El mensaje se imprime tal como se recibe, sin usar `decode()`, ya que los mensajes DNS son datos binarios y decodificarlos produciría errores.

```
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind((IP_VM, PORT))

while True:
    data, addr = sock.recvfrom(BUFFER_SIZE)
    print(data)
```

En este punto el resolver solo recibe mensajes pero no responde nada, por lo que `dig` muestra un timeout al consultar. Esto es lo esperado según el enunciado.

Paso 2. Parseo con dnslib

El segundo paso es parsear los mensajes DNS recibidos para poder trabajar con ellos. Se utiliza la función `DNSRecord.parse()` de `dnslib`, que convierte los bytes crudos en un objeto estructurado con atributos accesibles. La función `parse_dns` encapsula este parseo e imprime la información relevante del mensaje.

```
def parse_dns(data):
    msg = DNSRecord.parse(data)

    ancount = msg.header.a
    nscount = msg.header.auth
    arcount = msg.header.ar

    print(f"Dominio: {str(msg.q.qname)} | AN:{ancount} NS:{nscount} AR:
{arcount}")

    for rr in msg.rr:
        print(f"  A: {rr.rname} -> {rr.rdata}")

    for rr in msg.auth:
        print(f"  NS: {rr.rname} -> {rr.rdata}")

    for rr in msg.ar:
        ip = rr.rdata
        if QTYPE.get(rr.rtype) == "A":
            print(f"  AR: {rr.rname} -> {ip}")

    return msg
```

La función retorna el objeto `msg` para que pueda ser utilizado en los pasos siguientes. El objeto contiene los campos `msg.q` para la pregunta, `msg.rr` para la sección Answer, `msg.auth` para Authority y `msg.ar` para Additional.

Paso 3. Función resolve

La función `resolve` recibe el mensaje de consulta en bytes y lo envía a un servidor DNS, luego analiza la respuesta para decidir qué hacer. El algoritmo sigue el procedimiento iterativo descrito en el enunciado.

El procedimiento se divide en tres casos. El primer caso es cuando la respuesta contiene registros tipo A en la sección Answer, lo que significa que se encontró la IP del dominio.

En este caso se retornan los bytes de la respuesta completos, ya que `dig` espera recibir un mensaje DNS válido.

El segundo caso es cuando la respuesta contiene registros tipo NS en la sección Authority, lo que indica una delegación a otro Name Server. Primero se busca en la sección Additional si viene alguna IP asociada a esos NS. Si la IP está disponible se utiliza directamente. Si no hay IP en Additional, se toma el nombre de un NS de la sección Authority y se resuelve recursivamente llamando a la misma función `resolve` para obtener su IP. Una vez obtenida la IP del siguiente NS, se reenvía la query original a ese NS y se repite el proceso.

El tercer caso es cuando la respuesta no contiene A ni NS. En este se retorna `None` y se ignora la respuesta. Esta limitación impide manejar registros CNAME, NODATA o NXDOMAIN, lo cual se analiza en la sección de experimentos.

```
def resolve(query_data, ip_addr=ROOT_IP, debug=False):
    ns_name = "."
    domain = str(parse_dns(query_data).q.qname)
    while True:
        if debug:
            print(f"(debug) Consultando '{domain}' a {ns_name} con dirección
IP '{ip_addr}'")

        # a. enviar query al NS indicado
        sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        sock.settimeout(5)
        try:
            sock.sendto(query_data, (ip_addr, 53))
            data, _ = sock.recvfrom(BUFFER_SIZE)
        except socket.timeout:
            sock.close()
            return None
        sock.close()
        msg = parse_dns(data)

        for rr in msg.auth:
            if QTYPE.get(rr.rtype) == "NS":
                ns_name = str(rr.rdata)
                break

        # b. ¿hay registros A en Answer?
        for rr in msg.rr:
            if QTYPE.get(rr.rtype) == "A":
                return data

        # c. delegación a NS
        if msg.header.auth > 0:
```

```
            # buscar ip en additional
            next_ip = None
            for rr in msg.ar:
                if QTYPE.get(rr.rtype) == "A":
                    next_ip = str(rr.rdata)
                    break
            # si no hay, resolver el ns recursivamente
            if next_ip is None:
                for rr in msg.auth:
                    if QTYPE.get(rr.rtype) == "NS":
                        q = DNSRecord.question(ns_name)
                        ns_ip = resolve(bytes(q.pack()), ROOT_IP)
                        if ns_ip:
                            ns_msg = parse_dns(ns_ip)
                            if ns_msg.rr and QTYPE.get(ns_msg.rr[0].rtype) ==
"A":
                                next_ip = str(ns_msg.rr[0].rdata)
                            break
            # enviar query original al siguiente ns
            if next_ip:
                ip_addr = next_ip
                continue

        # ignorar otros tipos
        return None
```

Un detalle importante es que siempre se reenvía `query_data`, es decir la query original del cliente. No se crea una nueva query para el siguiente NS, sino que se reutiliza el mensaje tal cual lo recibió `dig`.

Paso 4. Modo debug

Se agregó un modo debug que muestra en cada paso de la resolución qué Name Server se está consultando. Se activa con el argumento `--debug` al ejecutar el resolver. En cada iteración del while se imprime el dominio consultado, el nombre del NS y la IP a la que se envía la query.

```
(debug) Consultando 'www.uchile.cl.' a . con dirección IP '198.41.0.4'
(debug) Consultando 'www.uchile.cl.' a 'cl.' con dirección IP '185.159.198.56'
(debug) Consultando 'www.uchile.cl.' a 'ns1.uchile.cl.' con dirección IP
'200.89.70.3'
```

La primera línea muestra la consulta al root server, la segunda al NS de Chile y la tercera al NS autoritativo de `uchile.cl` que finalmente tiene la IP del dominio.

Paso 5. Caché

El quinto paso agrega un caché al resolver. Este guarda los últimos 3 dominios más repetidos dentro de las últimas 20 consultas recibidas. Se utiliza una lista `history` para registrar los dominios consultados y un diccionario `cache` para almacenar las respuestas. La función `top3` calcula qué dominios están en el top 3 de frecuencia.

```
history = []
cache = {}
MAX_HISTORY = 20

def top3():
    counts = {}
    for d in history:
        counts[d] = counts.get(d, 0) + 1
    sorted_items = sorted(counts.items(), key=lambda x: x[1], reverse=True)
    return [d for d, _ in sorted_items][:3]
```

Cada consulta que recibe el resolver primero se busca en el caché. Si el dominio está, se retorna la respuesta guardada sin resolver de nuevo. Si no está, se agrega al historial, se resuelve normalmente y se guarda en el caché solo si el dominio está en el top 3.

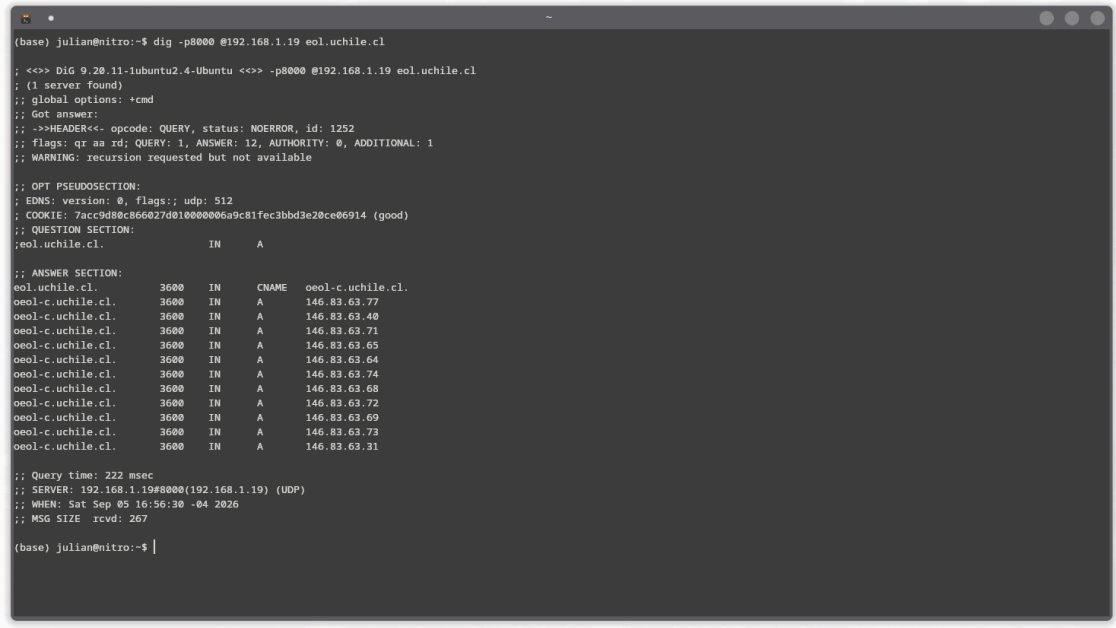
Hay un detalle importante al enviar una respuesta desde el caché. El mensaje DNS contiene un ID de transacción en los primeros 2 bytes que dig genera en cada consulta. Si se envía el mensaje cacheado con el ID viejo, dig lo rechaza con un error de ID mismatch. Por eso se copian los primeros 2 bytes de la query actual al mensaje cacheado antes de enviarlo.

```
if domain in cache:
    response = bytearray(cache[domain])
    response[0:2] = data[0:2]
    if debug:
        print(f"(debug) Usando caché para '{domain}'")
    sock.sendto(bytes(response), addr)
else:
    history.append(domain)
    if len(history) > MAX_HISTORY:
        history.pop(0)
    response = resolve(data, debug=debug)
    if response:
        if domain in top3():
            cache[domain] = response
        sock.sendto(response, addr)
```

Pruebas de funcionalidad

eol.uchile.cl

Se consultó `eol.uchile.cl` al resolver con `dig`. La primera consulta resuelve normalmente siguiendo la cadena de delegaciones desde el root server hasta el NS autoritativo. La IP obtenida es de la forma `146.83.63.X`.



```
(base) julian@nitro:~$ dig -p8000 @192.168.1.19 eol.uchile.cl

; <<>> DiG 9.20.11-1ubuntu2.4-Ubuntu <<>> -p8000 @192.168.1.19 eol.uchile.cl
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 1252
;; flags: qr aa rd; QUERY: 1, ANSWER: 12, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

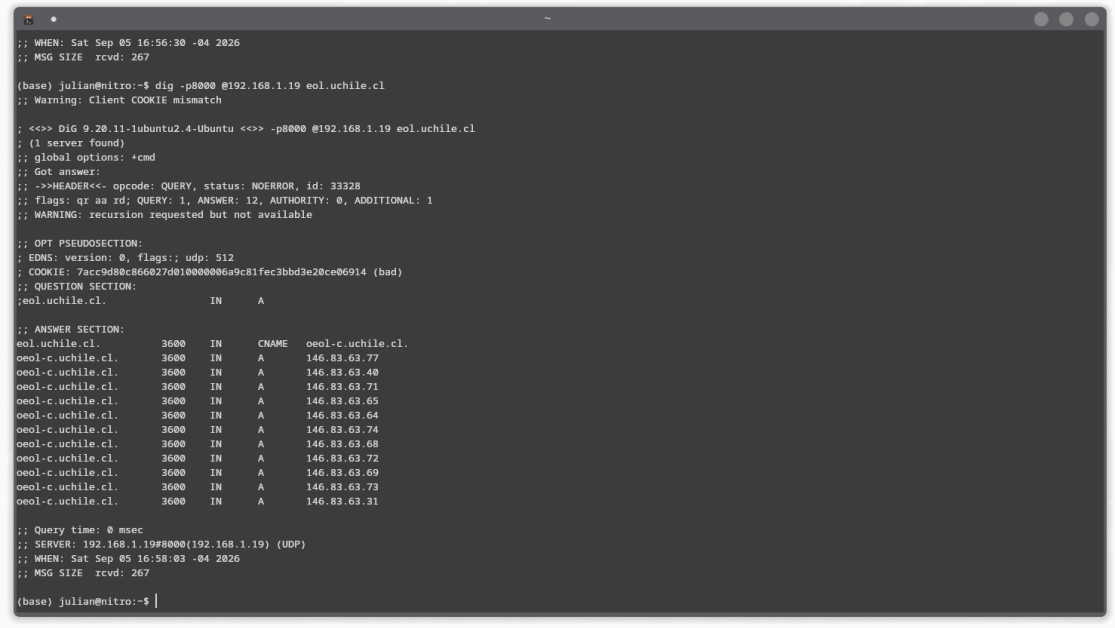
;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
; COOKIE: 7acc9d80c866027d010000006a9c81fec3bbd3e20ce06914 (good)
;; QUESTION SECTION:
;eol.uchile.cl.                 IN      A

;; ANSWER SECTION:
eol.uchile.cl.          3600    IN      CNAME   oeol-c.uchile.cl.
oeol-c.uchile.cl.       3600    IN      A       146.83.63.77
oeol-c.uchile.cl.       3600    IN      A       146.83.63.40
oeol-c.uchile.cl.       3600    IN      A       146.83.63.71
oeol-c.uchile.cl.       3600    IN      A       146.83.63.65
oeol-c.uchile.cl.       3600    IN      A       146.83.63.64
oeol-c.uchile.cl.       3600    IN      A       146.83.63.74
oeol-c.uchile.cl.       3600    IN      A       146.83.63.68
oeol-c.uchile.cl.       3600    IN      A       146.83.63.72
oeol-c.uchile.cl.       3600    IN      A       146.83.63.69
oeol-c.uchile.cl.       3600    IN      A       146.83.63.73
oeol-c.uchile.cl.       3600    IN      A       146.83.63.31

;; Query time: 222 msec
;; SERVER: 192.168.1.19#8000(192.168.1.19) (UDP)
;; WHEN: Sat Sep 05 16:56:30 -04 2026
;; MSG SIZE  rcvd: 267

(base) julian@nitro:~$
```

La segunda consulta al mismo dominio se responde desde el caché. Se puede ver que el tiempo de respuesta es significativamente menor.



```
;; WHEN: Sat Sep 05 16:56:30 -04 2026
;; MSG SIZE  rcvd: 267

(base) julian@nitro:~$ dig -p8000 @192.168.1.19 eol.uchile.cl
;; Warning: Client COOKIE mismatch

; <<>> DiG 9.20.11-1ubuntu2.4-Ubuntu <<>> -p8000 @192.168.1.19 eol.uchile.cl
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 33328
;; flags: qr aa rd; QUERY: 1, ANSWER: 12, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

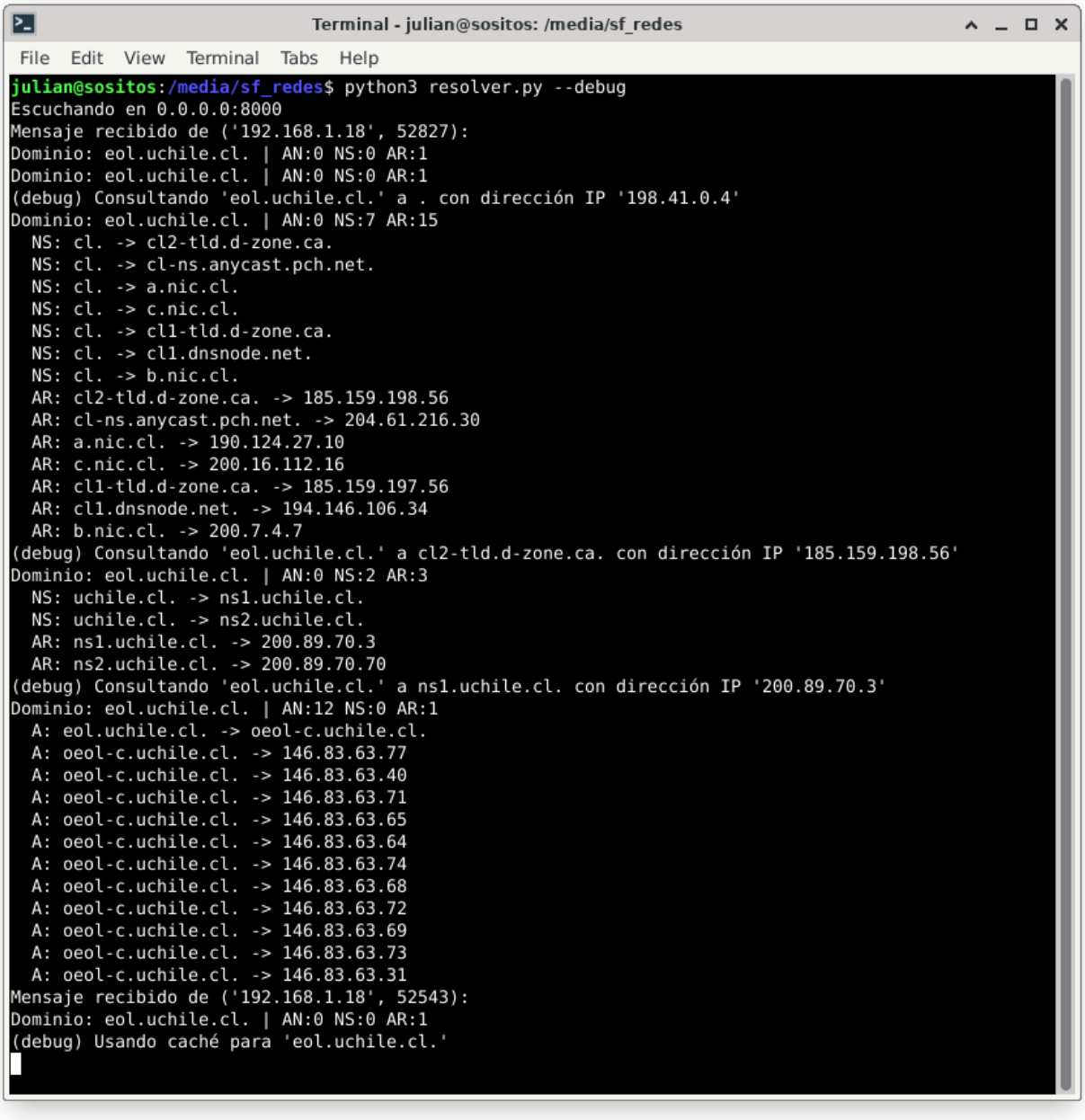
;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
; COOKIE: 7acc9d80c866027d010000006a9c81fec3bbd3e20ce06914 (bad)
;; QUESTION SECTION:
;eol.uchile.cl.                 IN      A

;; ANSWER SECTION:
eol.uchile.cl.          3600    IN      CNAME   oeol-c.uchile.cl.
oeol-c.uchile.cl.       3600    IN      A       146.83.63.77
oeol-c.uchile.cl.       3600    IN      A       146.83.63.40
oeol-c.uchile.cl.       3600    IN      A       146.83.63.71
oeol-c.uchile.cl.       3600    IN      A       146.83.63.65
oeol-c.uchile.cl.       3600    IN      A       146.83.63.64
oeol-c.uchile.cl.       3600    IN      A       146.83.63.74
oeol-c.uchile.cl.       3600    IN      A       146.83.63.68
oeol-c.uchile.cl.       3600    IN      A       146.83.63.72
oeol-c.uchile.cl.       3600    IN      A       146.83.63.69
oeol-c.uchile.cl.       3600    IN      A       146.83.63.73
oeol-c.uchile.cl.       3600    IN      A       146.83.63.31

;; Query time: 0 msec
;; SERVER: 192.168.1.19#8000(192.168.1.19) (UDP)
;; WHEN: Sat Sep 05 16:58:03 -04 2026
;; MSG SIZE  rcvd: 267

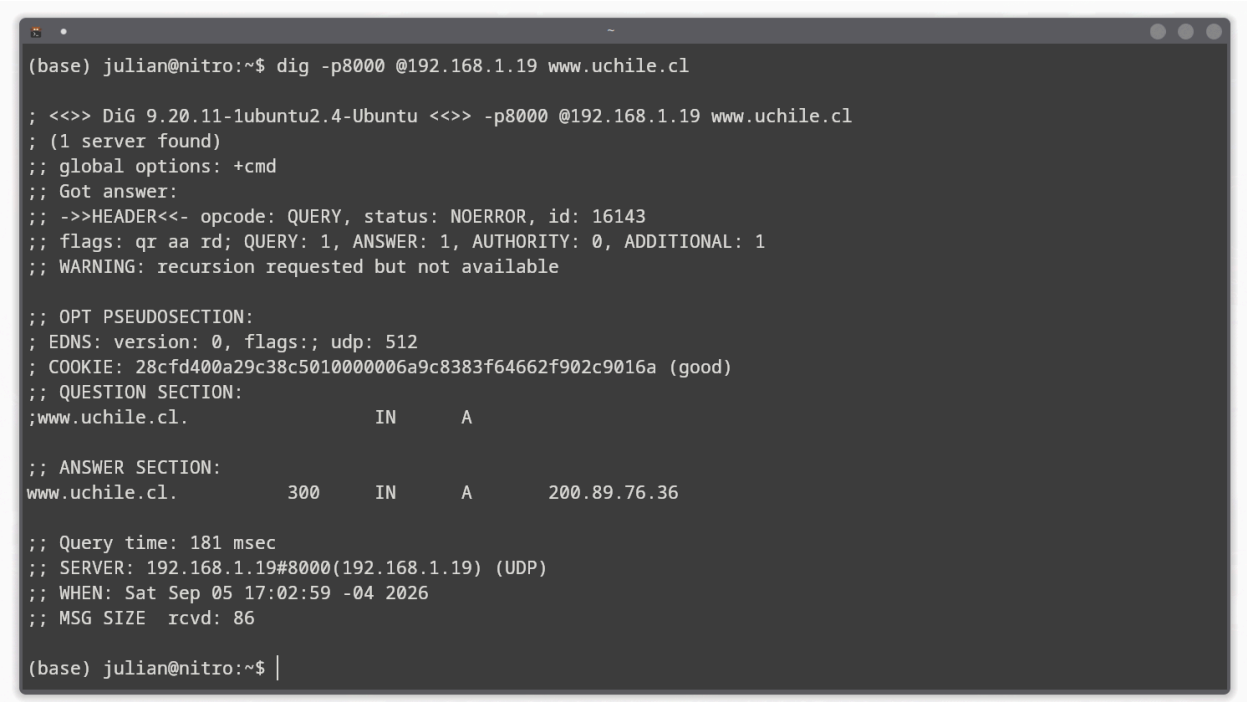
(base) julian@nitro:~$
```

El output del resolver muestra la cadena completa de delegaciones en la primera consulta y el uso del caché en la segunda.



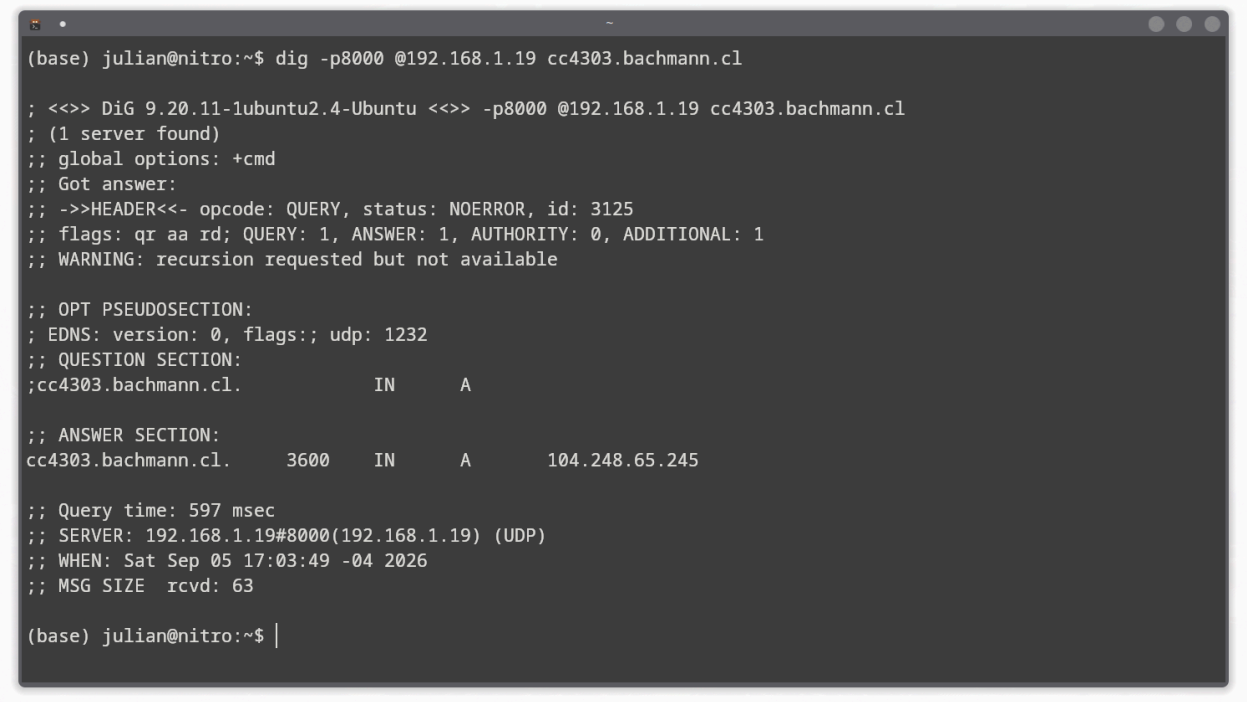
www.uchile.cl

La consulta a `www.uchile.cl` resuelve a `200.89.76.36`.



cc4303.bachmann.cl

La consulta a `cc4303.bachmann.cl` resuelve a `104.248.65.245`.

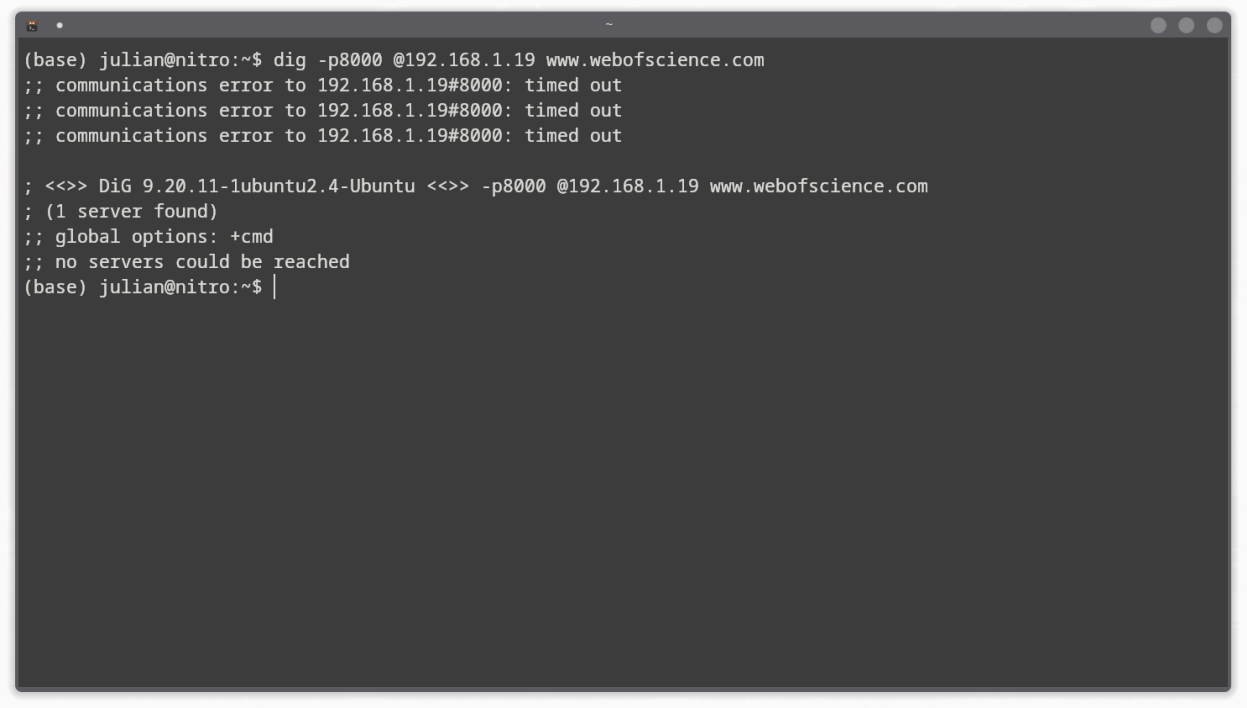


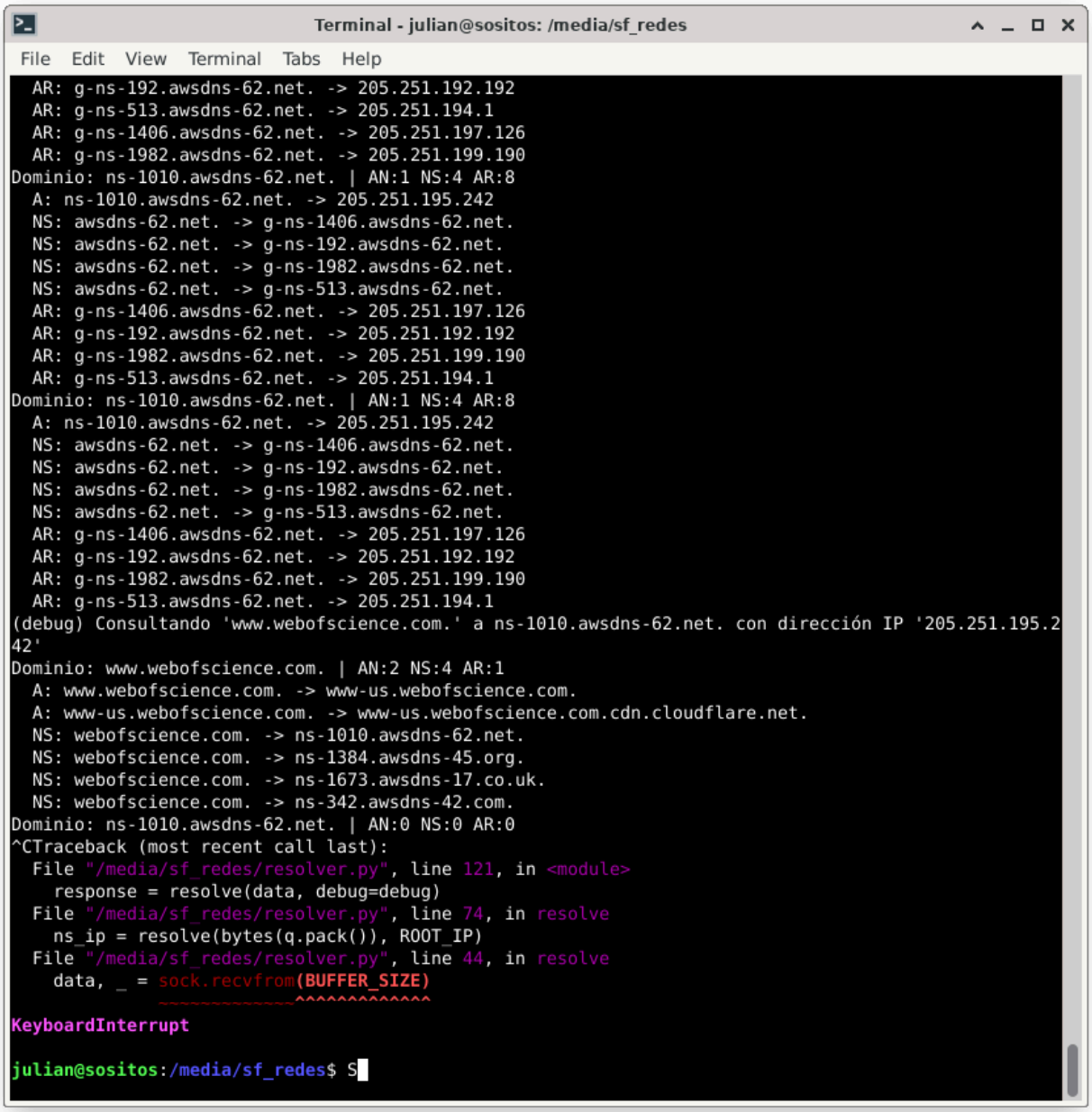
Experimentos

www.webofscience.com

Se intentó resolver `www.webofscience.com` con el resolver. El dominio no resuelve, el resolver entra en un loop infinito y `dig` termina con un timeout. Lo que sucede es que el NS autoritativo devuelve registros CNAME en vez de registros A. El dominio `www.webofscience.com` apunta a `www-us.webofscience.com`, que a su vez apunta a `www-us.webofscience.com.cdn.cloudflare.net`. Como el resolver solo busca registros tipo A en la sección Answer, no encuentra la IP y cae al bloque de Authority, donde resuelve el NS recursivamente y vuelve a preguntar, recibiendo los mismos CNAME, creando un loop infinito.

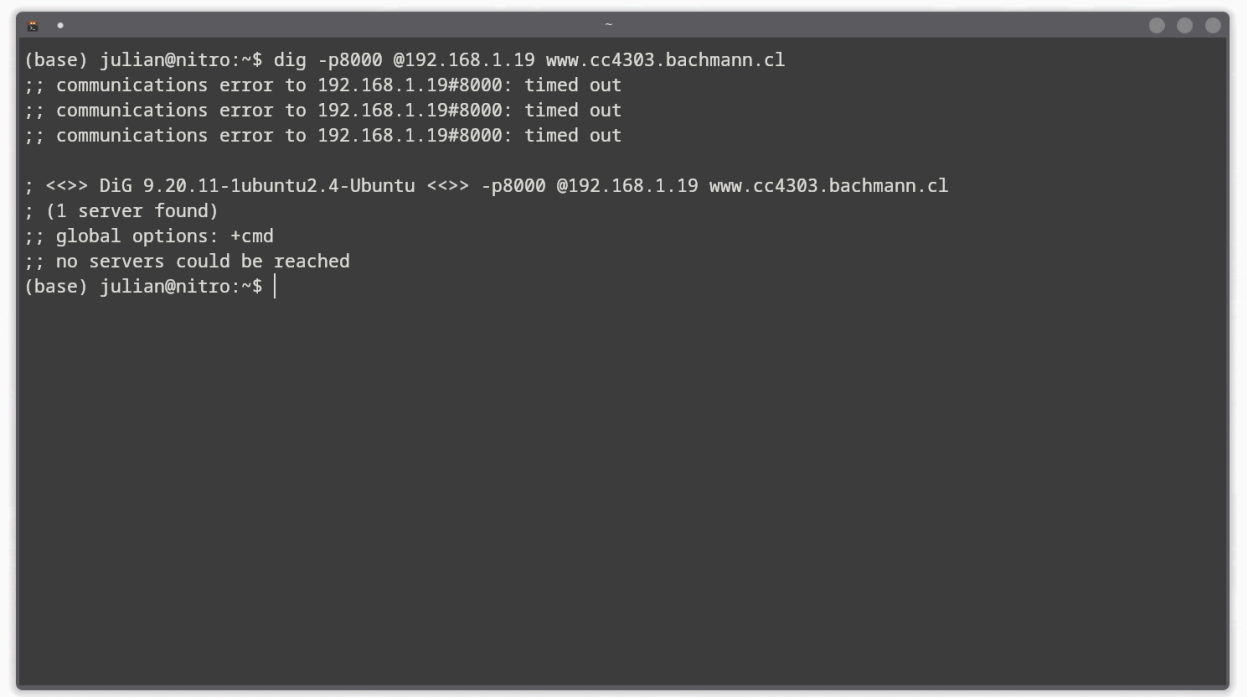
Para arreglar este problema habría que detectar registros CNAME en la sección Answer y seguir la cadena, haciendo una nueva query por el dominio alias hasta encontrar un registro A.



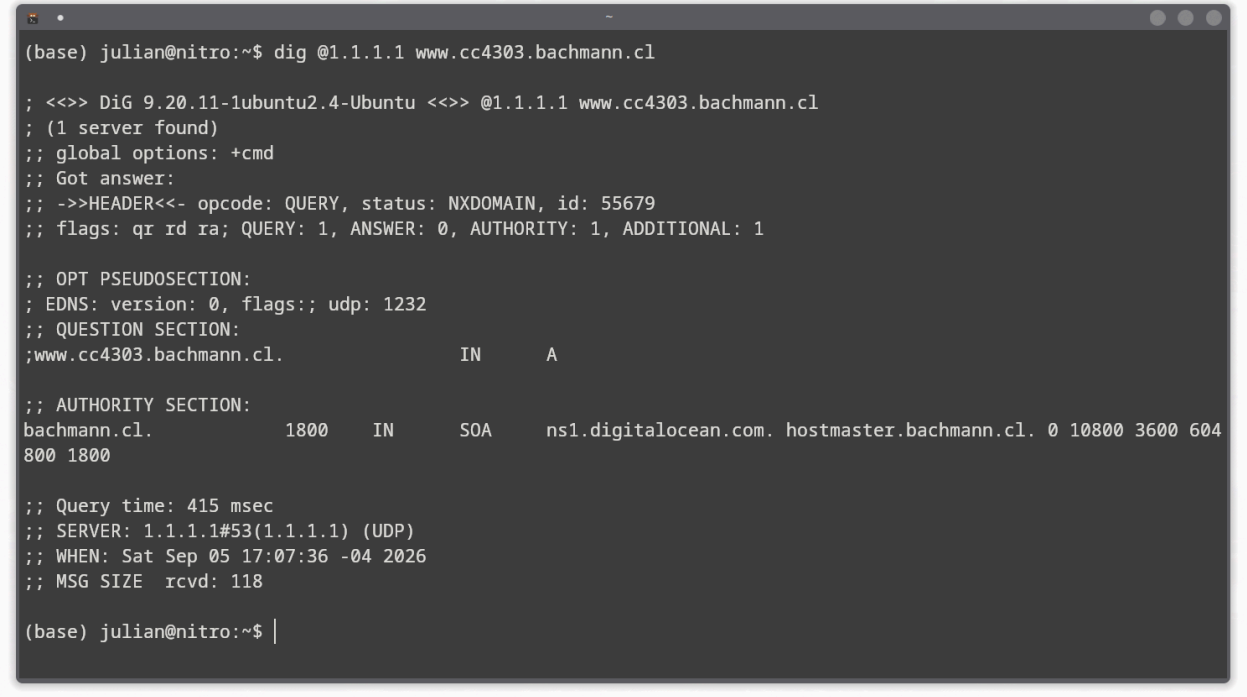


www.cc4303.bachmann.cl

Al consultar `www.cc4303.bachmann.cl` ocurre algo similar. El dominio tiene un registro CNAME que apunta a otro dominio. El resolver no sigue la cadena CNAME y no obtiene una IP.



Al comparar con la consulta a Cloudflare (`dig @1.1.1.1 www.cc4303.bachmann.cl`), se puede ver que Cloudflare sí resuelve el dominio porque sigue la cadena CNAME hasta encontrar el registro A.



Esto se explica porque los servidores DNS como Cloudflare implementan la resolución completa de cadenas CNAME, mientras que nuestro resolver solo busca registros A directos.

NS siempre iguales

Al realizar varias consultas a `www.uchile.cl` y observar el modo debug, se puede ver que los Name Servers consultados son siempre los mismos. El root server siempre delega a los NS de Chile, y los NS de Chile siempre delegan a los NS de `uchile.cl`. Esto sucede porque la jerarquía DNS es estática, los servidores raíz siempre apuntan a los mismos TLD y los TLD siempre apuntan a los mismos NS autoritativos para cada dominio.

Cómo ejecutar el código

El resolver se ejecuta en la máquina virtual desde la carpeta donde se encuentra `resolver.py`. Para iniciarlo se usa el siguiente comando.

```
python3 resolver.py --debug
```

El argumento `--debug` activa el modo debug que muestra las consultas internas. Sin ese argumento el resolver funciona igual pero sin imprimir las líneas de debug. El resolver queda escuchando en el puerto 8000 de todas las interfaces.

Para probar el resolver con `dig` se usa el siguiente formato.

```
dig -p8000 @IP_VM example.com
```

Repositorio de GitHub

El código de la actividad está disponible en el repositorio de GitHub al que el equipo docente tiene acceso. El enlace al repositorio es el siguiente.

[https://github.com/julianticucho/cc4303_tareas] (rama control1parte2)

Declaración de uso de IA

Se utilizó Gemini de Google como apoyo principalmente para entender la documentación de `dnslib`. Se uso el autocompletado que viene por defecto en VSCode. El código, las decisiones de diseño y las pruebas fueron escritas y validadas de forma manual para asegurar su correcto funcionamiento.